Chain-Guided LLM to Generate candidate VFCS Prompt Template

1. Contract Source Code:

Please provide the source code of the smart contract to be analyzed for initial review and analysis.

2. Solidity Version:

Please specify the Solidity version used by the smart contract to identify potential vulnerabilities unique to that version.

3. Initial Analysis:

Conduct an initial analysis to identify key functions and private functions.

- Identify key elements influencing the likelihood of vulnerabilities (e.g., access control and state variables).
- Assess the version impact on the smart contract and filter out irrelevant vulnerabilities.
- Hypothesize potential vulnerabilities and document relevant key variables.

4. Enhanced Analysis with Static Tools:

Enhance the initial analysis by incorporating insights from static analysis tools (e.g., Slither).

- Obtain control flow graphs, call graphs, and ABI data to support data-driven evaluation.
- Revisit and validate findings from the initial analysis (Phase 3).
- Confirm or rule out specific vulnerabilities and identify critical function-variable interactions.

5. Synthesize Findings and Generate Candidate VFCSs:

Synthesize findings from previous phases to generate candidate VFCSs.

- Leverage examples of vulnerabilities identified in similar contracts to generate candidate VFCSs with contextual understanding.
- Use few-shot prompting with structured templates to guide the LLM in generating VFCS likely leading to vulnerabilities.
- Output three specific candidate VFCSs, each representing a plausible path to a vulnerability for further validation and fuzzing.

6. Regularization and Fuzzer Integration:

Regularize the candidate VFCS to ensure compatibility with fuzzing tools.

- Clean up any syntactical inconsistencies, such as stray symbols and format misalignments.
- Normalize parameters and function calls to match the input requirements of the target fuzzer.
- Use an automated transformation script to adapt the VFCSs into a format directly usable by fuzzing tools, ensuring no discrepancies in function parameters or call patterns.

7. Output Optimized Candidate VFCSs:

Output the optimized candidate VFCSs, ensuring their reliability and efficiency in fuzz testing applications.